

Q1 (a) Explain the structure of a C program C program is written in different parts, and each part has its own role. Usually, the program starts with header files. Header files are written using `#include`. They are used so that we can use ready-made functions like `printf` and `scanf`. For example, `#include` is commonly used for input and output work. After header files, sometimes we define constants or write global variables. These are optional and used only when needed. Then comes the main part of the program, which is the `main()` function. Execution of the program always starts from the `main()` function. Inside the `main()` function, we write statements like variable declaration, input, processing, and output. Each statement usually ends with a semicolon. At the end of the program, `return 0;` is written. This shows that the program finished properly. Apart from `main()`, a C program can also have user-defined functions. These functions are written either before or after the `main()` function and are called when required. Overall, the structure of a C program is simple and easy to follow when written step by step.

Q1 (b) Explain input/output functions (`printf`, `scanf`) In C programming, input and output are done using functions. Most commonly used output function is `printf`. It is used to display messages or values on the screen. For example, if we want to show a message, we write `printf("name");`. It can also print values of variables using format symbols like `%d` for integers and `%f` for float values. To taking input from the user, `scanf` function is used. This function reads data entered through the keyboard. While using `scanf`, we must give the address of the variable using the `&` symbol. For example, `scanf("test", &num);` stores the entered value into variable `num`. Both `printf` and `scanf` are defined inside `stdio.h`, so this header file must be included. These functions help the program to interact with the user by taking input and showing output, which is very important in most C programs.